



RAISE-Bot

— Hands-on ROS 2, AI Agents, and Greenhouse Robot Inspection



Building the Day-2 Dataset

Student edition — how the robot taught itself to pick

Professor Anis Koubaa • 14–16 July 2026



Husky A200
Base

The goal: a dataset that teaches by showing

A VLA learns by **imitation**: show it examples of

*“this is what the camera saw,
and this is the action taken”*

and it learns to copy the mapping.

- No rewards. No trial-and-error. **Not RL.**
- The dataset **is** the teacher — its quality decides everything.

The Day-2 thread

What you **record** in the morning
is the **brain** you command
in the afternoon.

One training frame = see + be + do (+ the words)

Every $\frac{1}{10}$ s the recorder snapshots four things:

Field	Shape	Meaning	Comes from
<code>observation.images.wrist</code>	$224 \times 224 \times 3$	what the robot sees	<code>/wrist_camera/image_raw</code>
<code>observation.state</code>	7 floats	where the robot is	<code>/joint_states</code>
<code>action</code>	7 floats	what it is told to do	the commanded joint targets
<code>task</code>	string	the words	"pick the red tomato"

The model learns $(\text{image}, \text{task}, \text{state}) \rightarrow \text{action}$.

7 floats = 6 UR5e joint angles + 1 gripper value. **action = the command**, not the measured pose — cloning wants the *intent* at each timestep.

Trick 1 — grasping by decree (`grasp_server`)

A small ROS 2 node that makes grasping **deterministic**:

- gripper **commanded closed** within 15 cm of a registered tomato $\Rightarrow$ the tomato **attaches** (teleported to follow the tool)
- commanded **open** $\Rightarrow$ **released**
- watches the **command** topic, not the measured angle: a tomato stalls the knuckle before “closed”, so command = intent.

Why not real friction?

Fighting physics gives you flaky grasps and flaky demos. For *teaching*, a grasp that works 100% of the time beats a realistic one that works 60%.

Works identically for the scripted expert, a teleoperating human, **and the trained VLA** — anyone who closes the gripper grasps.

Trick 2 — no IK? Spawn the tomato where the hand lands

Hard question: which joint angles put the gripper *at the tomato*? (That's inverse kinematics — we don't have it.)

Easy question: where does the gripper end up if I move the joints to a pose I chose? (That's just reading the robot's own TF.)

- 1 move the arm to a comfortable “grasp pose” (chosen joint angles)
- 2 read **where the tool actually is** in the world
- 3 **spawn the tomato exactly there**

Every grasp is reachable **by construction**. Zero IK.
(Two such poses — LEFT and RIGHT — give the task its spatial variety.)

Trick 3 + the episode pipeline

Trick 3 — gravity-off tomatoes. Spawned with `<gravity>false</gravity>`, they float where placed — no table, carried and released anywhere. Red = ripe, green = distractor.

One episode of `05_auto_demonstrate.py` (of 50)

```
1 SPAWN    red + green tomato at the two grasp points (red side alternates!)
2 HOME     arm to start pose, gripper open
3 RECORD   SCAN (look above left; green? pan right) → descend →
           close (grasp!) → lift → swing → release    ... at 10 Hz
4 VERIFY   /grasp/state must name the RED tomato | else the take is discarded
5 CLEAN    remove both tomatoes, next episode
```

Step 4 is the quality gate: a failed grasp teaches failure — it never enters the dataset.

The reference dataset, by the numbers

Repo id	<code>raiseschool/raise_ripeness_sort_ref</code>
Instruction	“pick the red tomato”
Episodes	50 (25 red-left + 25 red-right)
Frames / episode	~86 (avg; ~8.6 s at 10 Hz)
Total frames	4325 training examples
Image	224 × 224 RGB wrist camera
State / action	7 floats (6 joints + gripper)
Format	LeRobot v3.0 (parquet + media + metadata)
Recording time	~25 min, hands-free, headless sim

Headless = Gazebo runs *without a GUI* (`headless:=true`); the wrist camera still renders on the GPU. You can record datasets on a server.

Golden rule: replay before you train

The cheapest data check: **play the recorded actions back** into the sim and watch.

```
06_d3 --task C --team ref --episode 0 --spawn # open-loop replay
```

- **Smooth motion ending in a clean pick** → the data is good; train.
- **Jerky / wrong / no grasp** → fix the *demos*, not the model.

Garbage in, garbage out — and replay is where you catch the garbage, *before* spending GPU-hours on training.

The best bug of the school: the model was blind

Adding greenhouse context dropped success to **50% — exactly chance**, and neither more data (29→50 episodes) nor more training (3000→6000 steps) helped.

The cause: at every decision pose the tomatoes were **hidden behind the gripper**. “Which side is red?” was **not in the observation** — no optimizer learns what the camera cannot see.

The fix was choreography, not the model: a **SCAN** — look above a spot; red → descend, green → pan on. Every frame becomes decidable. Score: 40 → **100/100**.

The rule to remember

When a policy is stuck at **chance**, audit the **observation** before you scale the data: capture exactly what the camera sees at decision time, with the scene set up.

This is *active perception* — real robots look before they act, and now so does ours.

Takeaways

- 1 A VLA dataset is just **(image, words, state)** → **action** tuples — 4325 here.
- 2 The teacher can be a **script**: imitation copies whoever demonstrates, and scripts demonstrate perfectly.
- 3 Simulators **bend to teaching**: grasp-by-decree, no-IK spawn-at-tool, gravity-off props — simplify the mechanics, *never* the learning.
- 4 Build in **variation** (red left/right) or the model memorizes instead of understanding.
- 5 **Verify every episode** and **replay before training** — data quality beats model size.
- 6 Every bug (inference pipeline, chunk queue, **observability**) was found by **running the system**, not reading. Build, run, measure.